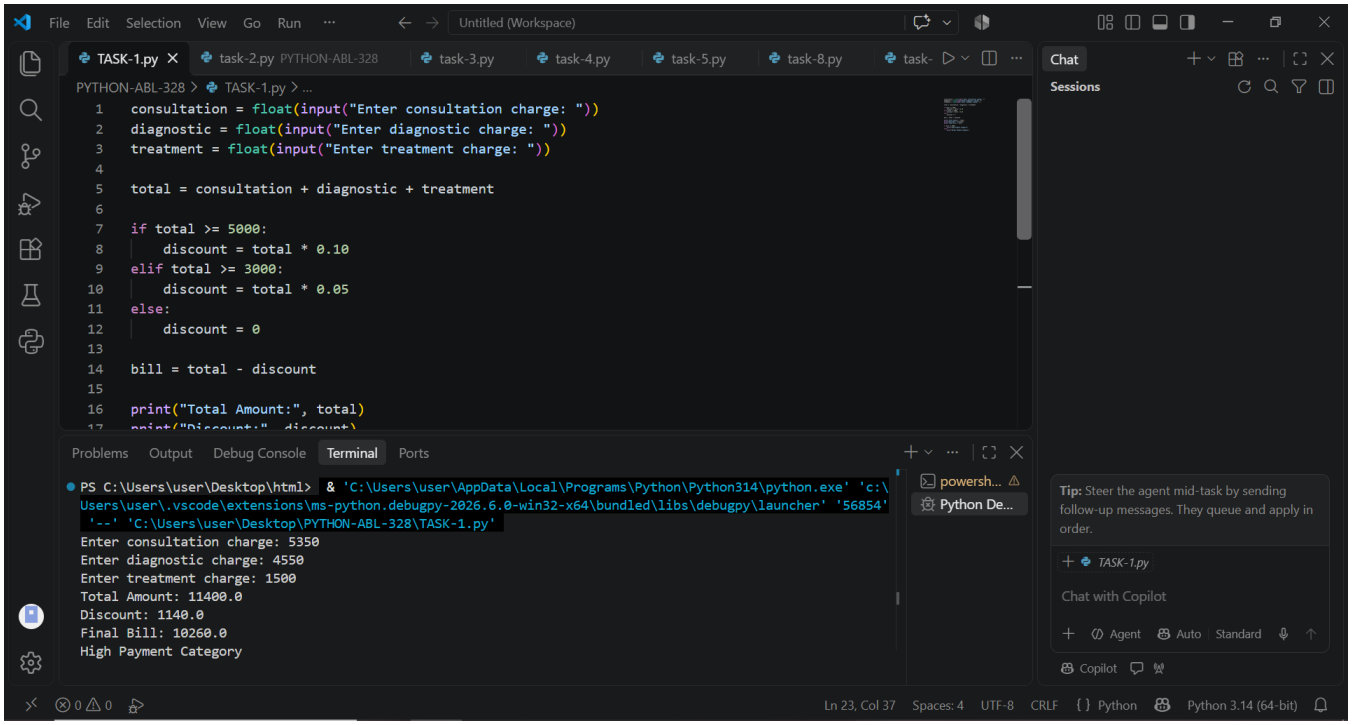
Python Activity-Based Learning

Task-1: Hospital Billing and Patient Classification



1. Problem Statement

Develop a program to calculate a patient bill from consultation, diagnostic, and treatment charges. Apply a concession based on the total and classify the payment category.

2. Objectives

- Accept the three charge amounts.
- Calculate the total bill and applicable discount.
- Display the final bill and payment category.

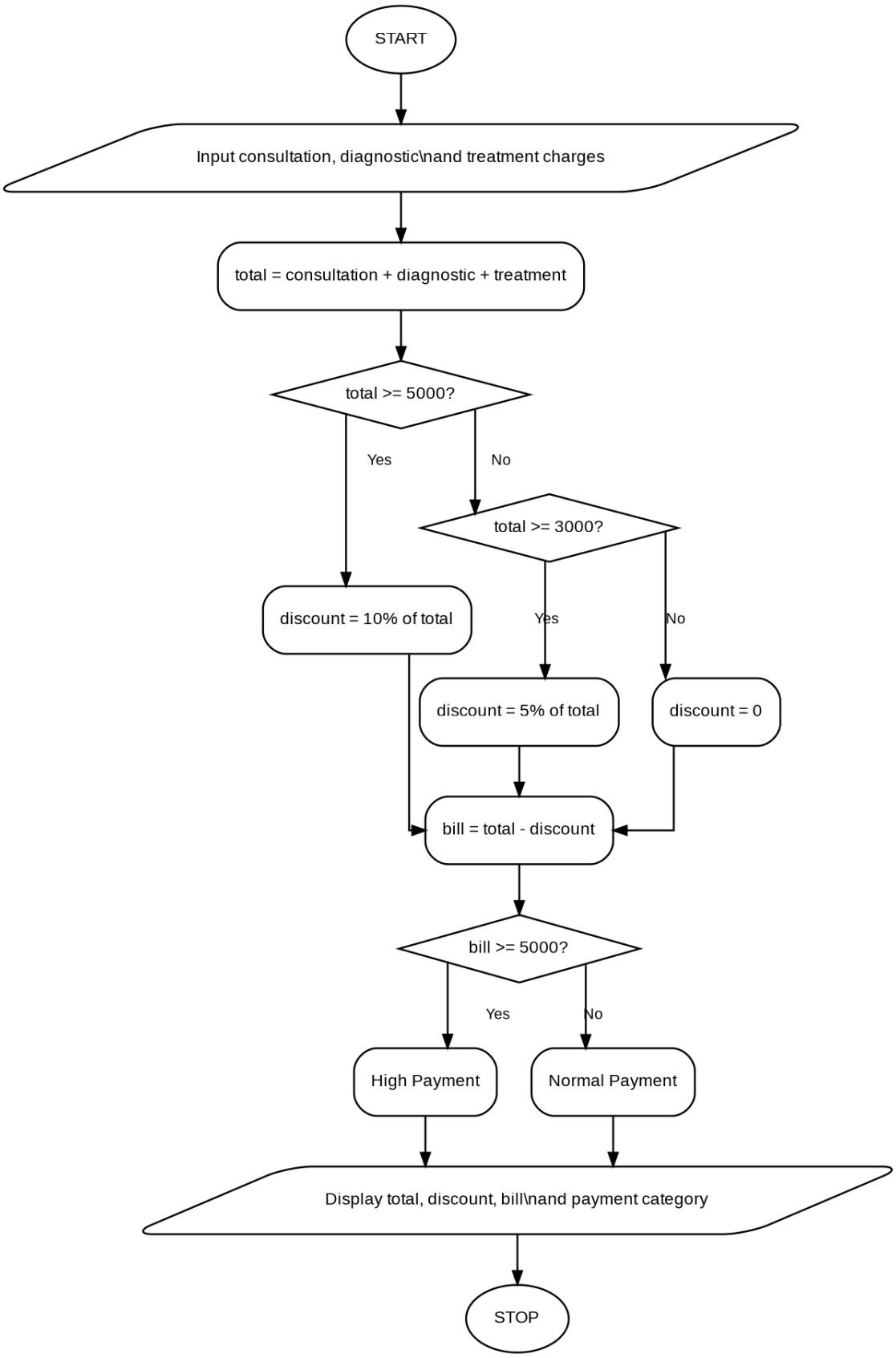
3. Assumptions

- All entered charges are non-negative numbers.
- A 10% discount applies when the total is at least ₹5,000.
- A 5% discount applies when the total is at least ₹3,000 but below ₹5,000; otherwise there is no discount.
- A final bill of ₹5,000 or more is classified as High Payment; otherwise Normal Payment.

4. Algorithm

1. Start.
2. Read consultation, diagnostic, and treatment charges.
3. Add the charges to get the total.
4. Choose the discount using the stated thresholds.
5. Subtract the discount from the total.
6. Display total, discount, and final bill.
7. Classify and display the payment category.
8. .Stop

5. Flowchart


START
Input consultation, diagnostic\nand treatment charges
total = consultation + diagnostic + treatment
total >= 5000?
Yes
No
total >= 3000?
discount = 10% of total
Yes
No
discount = 5% of total
discount = 0
bill = total - discount
bill >= 5000?
Yes
No
High Payment
Normal Payment
Display total, discount, bill\nand payment category
STOP

6. Pseudocode

```
START
INPUT consultation, diagnostic, treatment
total ← consultation + diagnostic + treatment
IF total >= 5000 THEN discount ← total × 0.10
ELSE IF total >= 3000 THEN discount ← total × 0.05
ELSE discount ← 0
bill ← total – discount
DISPLAY total, discount, bill
IF bill >= 5000 DISPLAY "High Payment" ELSE DISPLAY "Normal Payment"
STOP
```

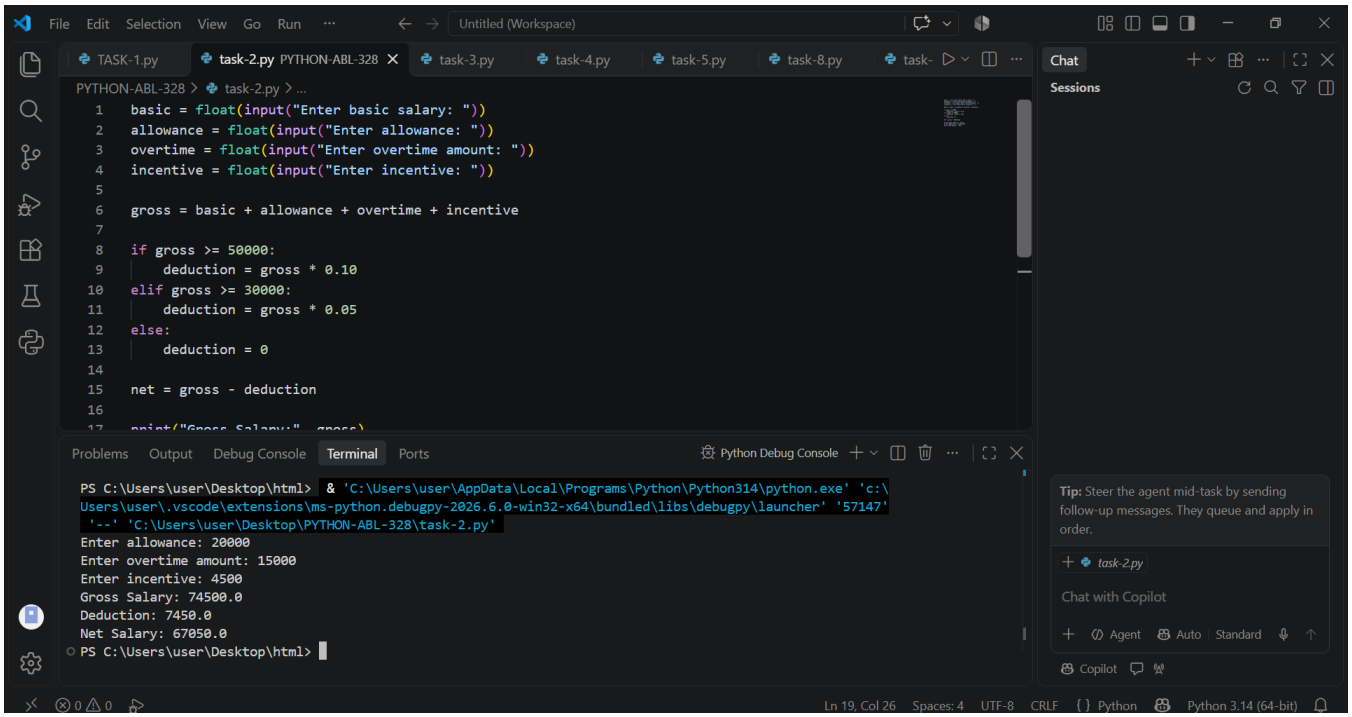
7. Test Cases

Input	Expected result
1000, 500, 1000	Total 2500; discount 0; bill 2500; Normal Payment
2000, 1000, 1000	Total 4000; discount 200; bill 3800; Normal Payment
3000, 1500, 1000	Total 5500; discount 550; bill 4950; Normal Payment
4000, 1000, 1000	Total 6000; discount 600; bill 5400; High Payment

8. Conclusion

The program calculates the patient bill, applies the assumed concession rules, and classifies the final payment amount.

Task-2: Employee Payroll and Deduction Analysis



1. Problem Statement

Create a payroll program that calculates gross salary from basic salary, allowance, overtime, and incentive, then applies a deduction and displays net salary.

2. Objectives

- Read salary components.
- Calculate gross salary.
- Apply a deduction based on gross salary.
- Display gross salary, deduction, and net salary.

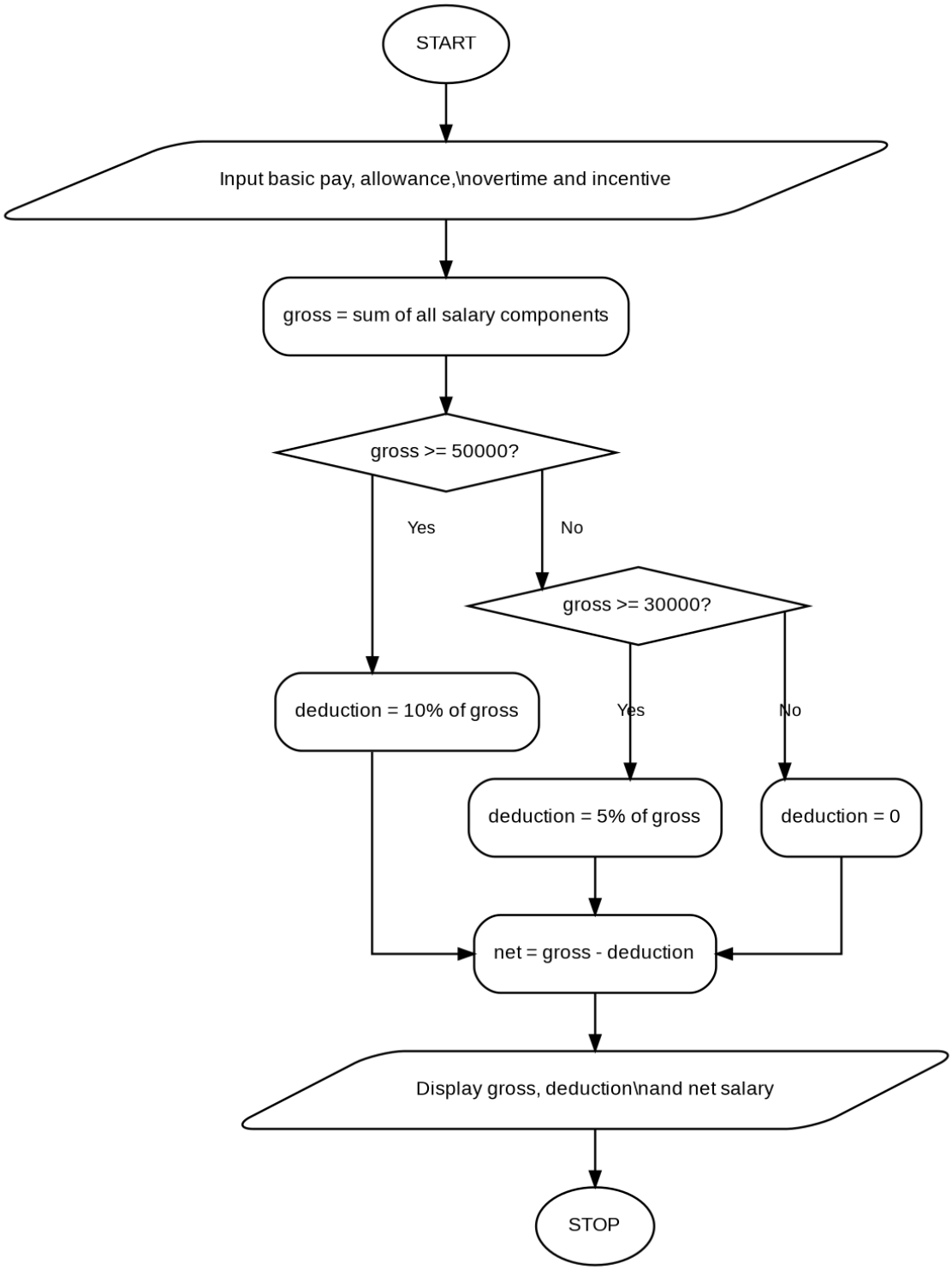
3. Assumptions

- All salary components are non-negative.
- Deduction is 10% for gross salary of ₹50,000 or more.
- Deduction is 5% for gross salary from ₹30,000 to below ₹50,000.
- No deduction is applied below ₹30,000.

4. Algorithm

9. Start.
10. Read basic salary, allowance, overtime, and incentive.
11. Add all components to calculate gross salary.
12. Select the deduction rate using the thresholds.
13. Calculate deduction and net salary.
14. Display the salary statement.
15. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT basic, allowance, overtime, incentive
gross ← basic + allowance + overtime + incentive
IF gross >= 50000 THEN rate ← 0.10
ELSE IF gross >= 30000 THEN rate ← 0.05
ELSE rate ← 0
```

```
deduction ← gross × rate
net ← gross – deduction
DISPLAY gross, deduction, net
STOP
```

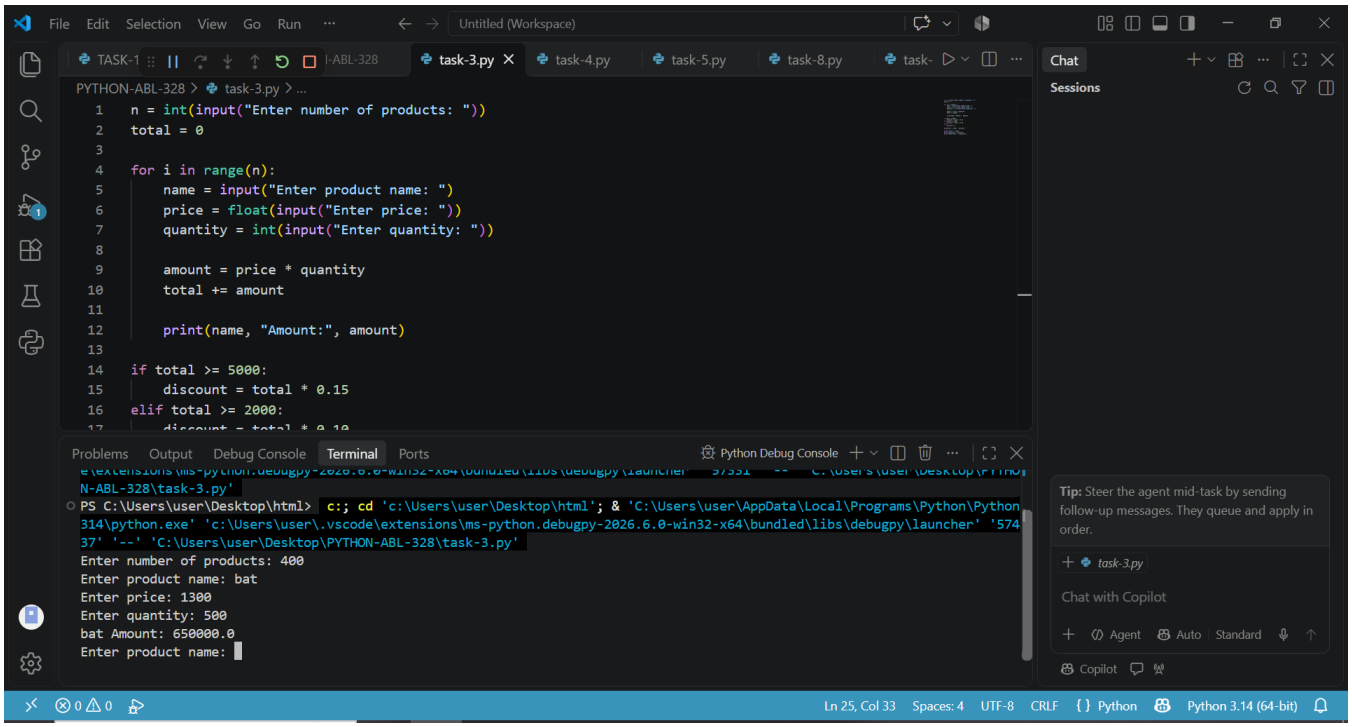
7. Test Cases

Input	Expected result
20000, 3000, 1000, 1000	Gross 25000; deduction 0; net 25000
25000, 3000, 1000, 2000	Gross 31000; deduction 1550; net 29450
40000, 5000, 3000, 4000	Gross 52000; deduction 5200; net 46800

8. Conclusion

The program produces a basic salary statement by combining salary components and applying the assumed progressive deduction slabs.

Task-3: Retail Billing with Multiple Discount Rules



1. Problem Statement

Develop a billing program that accepts multiple product names, prices, and quantities, calculates item amounts and total purchase value, and applies a threshold discount.

2. Objectives

- Accept details for multiple products.
- Calculate each item amount and the subtotal.
- Apply a discount based on the subtotal.
- Display the final bill.

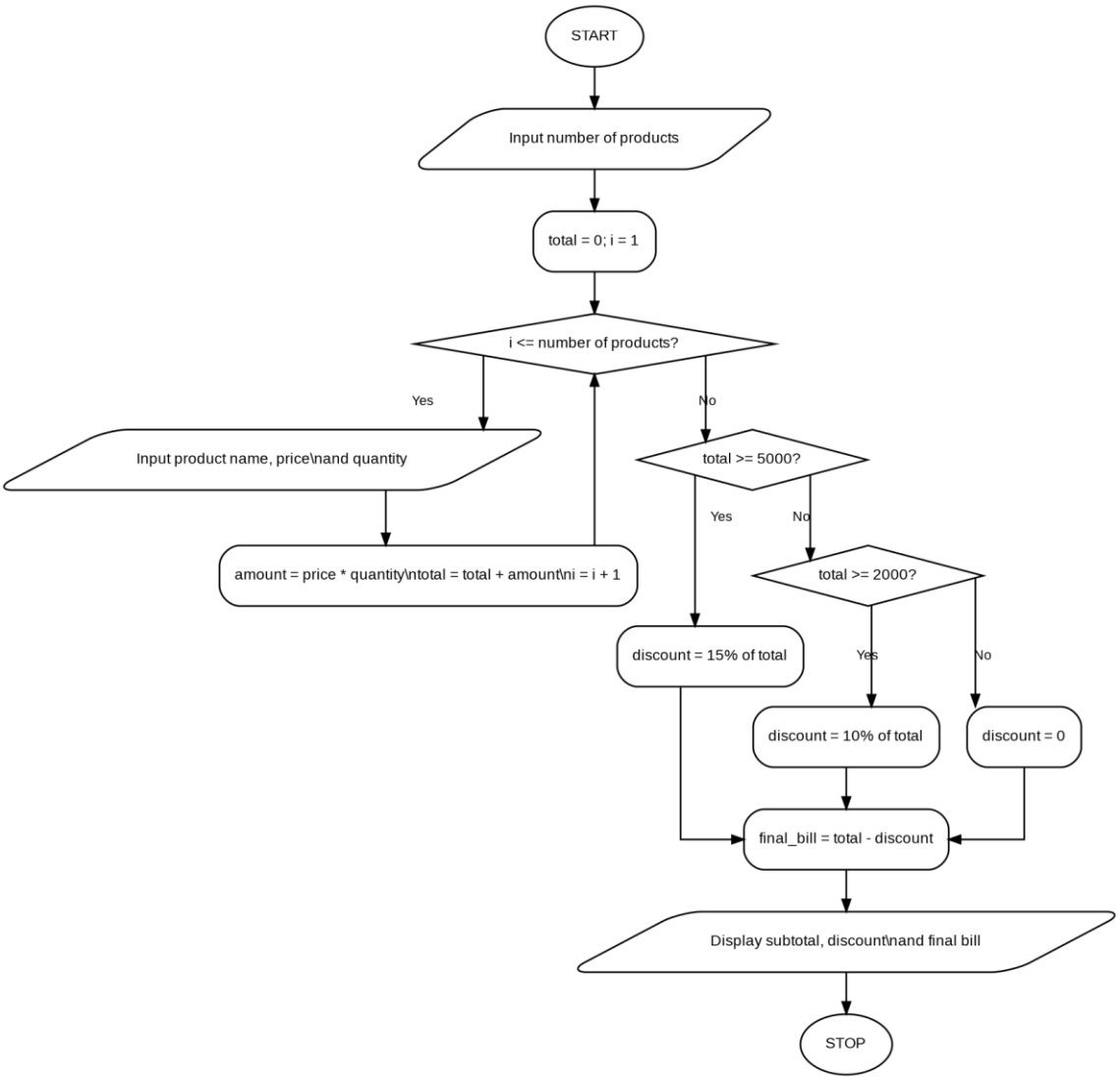
3. Assumptions

- The number of products is a positive integer.
- Prices and quantities are non-negative.
- A 15% discount applies to totals of ₹5,000 or more.
- A 10% discount applies to totals from ₹2,000 to below ₹5,000.
- No discount applies below ₹2,000; taxes are not included.

4. Algorithm

16. Start.
17. Read the number of products.
18. Set total to zero.
19. Repeat for each product: read name, price, and quantity; calculate amount; add it to total; display amount.
20. Select the discount rate using the total.
21. Calculate discount and final bill.
22. Display subtotal, discount, and final bill.
23. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
total ← 0
FOR each product from 1 to n
  INPUT name, price, quantity
  amount ← price × quantity
  total ← total + amount
  DISPLAY name, amount
END FOR
IF total >= 5000 discount ← total × 0.15
ELSE IF total >= 2000 discount ← total × 0.10
ELSE discount ← 0
final_bill ← total – discount
DISPLAY total, discount, final_bill
STOP
```

7. Test Cases

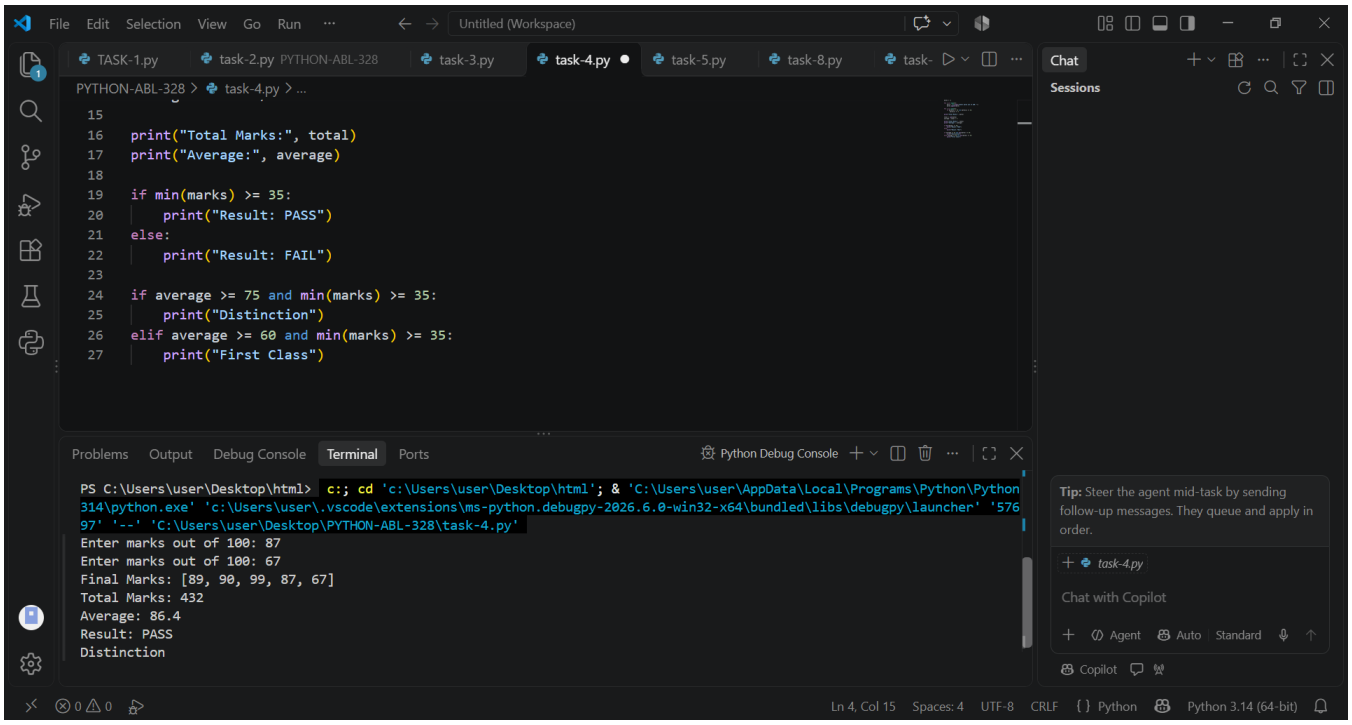
Input	Expected result

1 item: price 1000, quantity 1	Subtotal 1000; discount 0; final 1000
2 items: amounts 1200 and 1000	Subtotal 2200; discount 220; final 1980
2 items: amounts 3000 and 2500	Subtotal 5500; discount 825; final 4675

8. Conclusion

The program uses a loop to process a shopping basket and applies a discount according to the total purchase amount.

Task-4: Examination Result and Grace-Mark Validation



1. Problem Statement

Create a program to accept marks in five subjects, apply a limited grace-mark rule to eligible near-pass marks, and determine the result and classification.

2. Objectives

- Read marks for five subjects.
- Apply the assumed grace-mark rule.
- Calculate total and average marks.
- Determine pass/fail status and classification.

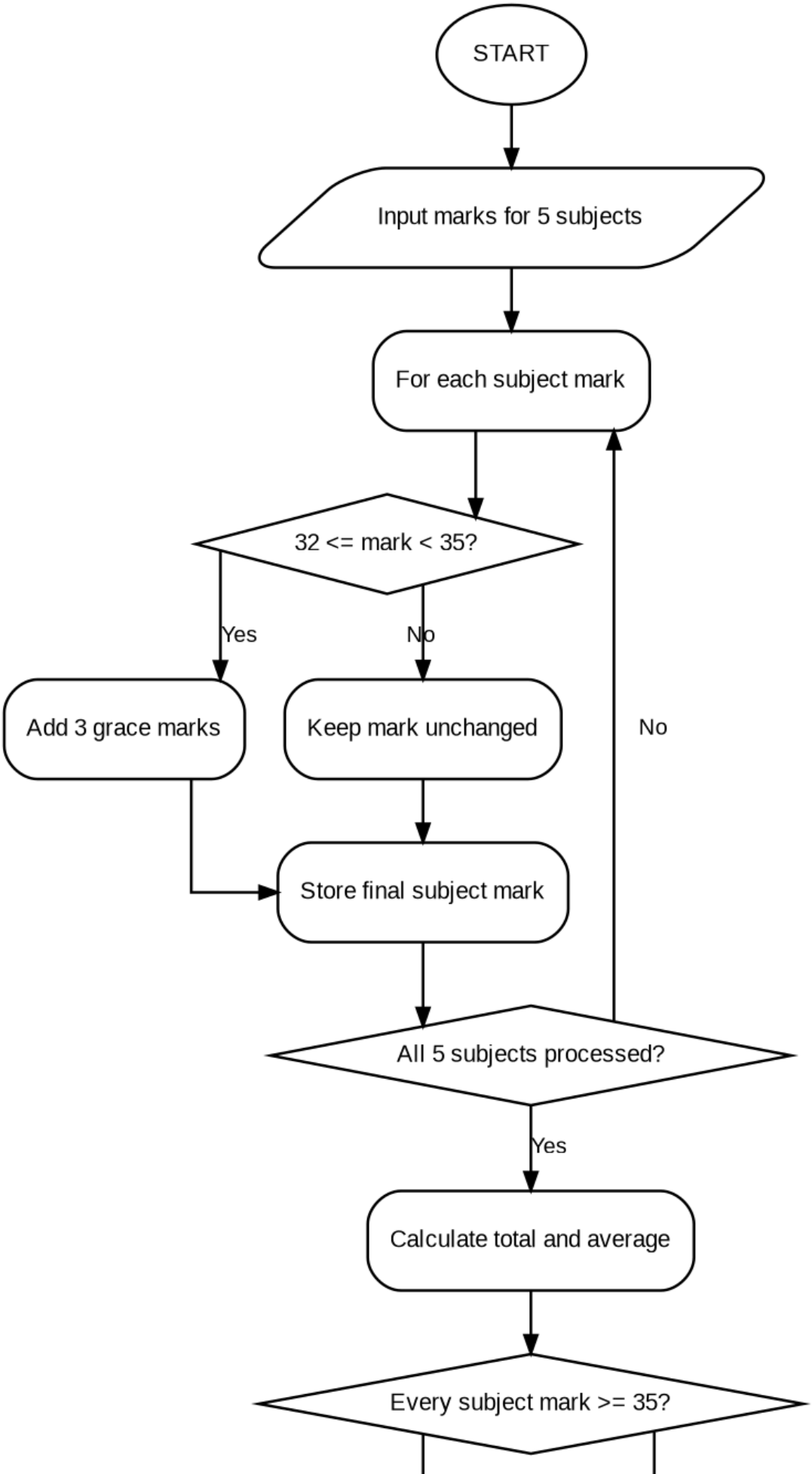
3. Assumptions

- There are exactly five subjects, each out of 100.
- A subject mark from 32 through 34 receives 3 grace marks.
- Grace marks must not make a mark exceed 100.
- A student passes only if every subject mark is at least 35 after moderation.
- Classification is shown only for a passing student: distinction at 75 average, first class at 60 average, otherwise pass.

4. Algorithm

24. Start.
25. Read five subject marks.
26. For each mark, if it is from 32 to 34, add 3.
27. Calculate total and average.
28. Check whether every subject mark is at least 35.
29. If passed, determine classification from average.
30. Display moderated marks, total, average, and result.
31. Stop.

5. Flowchart


START
Input marks for 5 subjects
For each subject mark
32 <= mark < 35?
Yes
No
Add 3 grace marks
Keep mark unchanged
Store final subject mark
No
All 5 subjects processed?
Yes
Calculate total and average
Every subject mark >= 35?

6. Pseudocode

```
START
marks ← empty list
REPEAT 5 times: INPUT mark; IF 32 <= mark < 35 THEN mark ← MIN(mark + 3, 100); append mark
total ← SUM(marks)
average ← total / 5
IF every mark >= 35 THEN
  DISPLAY "PASS"
  IF average >= 75 DISPLAY "Distinction"
  ELSE IF average >= 60 DISPLAY "First Class"
  ELSE DISPLAY "Pass"
ELSE DISPLAY "FAIL"
DISPLAY marks, total, average
STOP
```

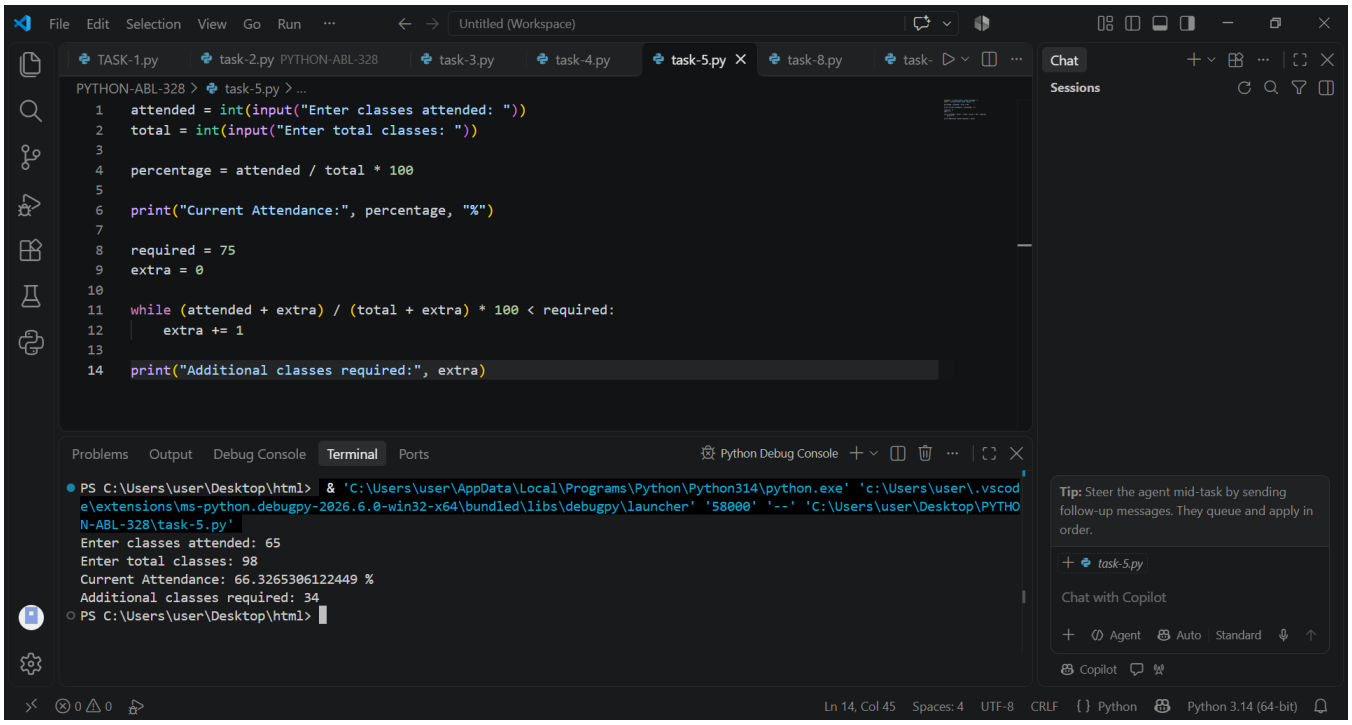
7. Test Cases

Input	Expected result
34, 40, 50, 60, 70	34 becomes 37; total 257; average 51.4; PASS
31, 40, 50, 60, 70	31 remains 31; FAIL
80, 75, 90, 85, 80	Total 410; average 82; Distinction

8. Conclusion

The program demonstrates list processing, conditional grace-mark application, aggregation, and result classification. The grace policy is an example and must match the faculty rule.

Task-5: Attendance Recovery Computation



1. Problem Statement

Write a program to calculate current attendance percentage and find the minimum number of additional consecutive classes that must be attended to reach 75%.

2. Objectives

- Read attended and total classes.
- Calculate current attendance percentage.
- Use a loop to find the minimum extra classes needed.
- Display the result.

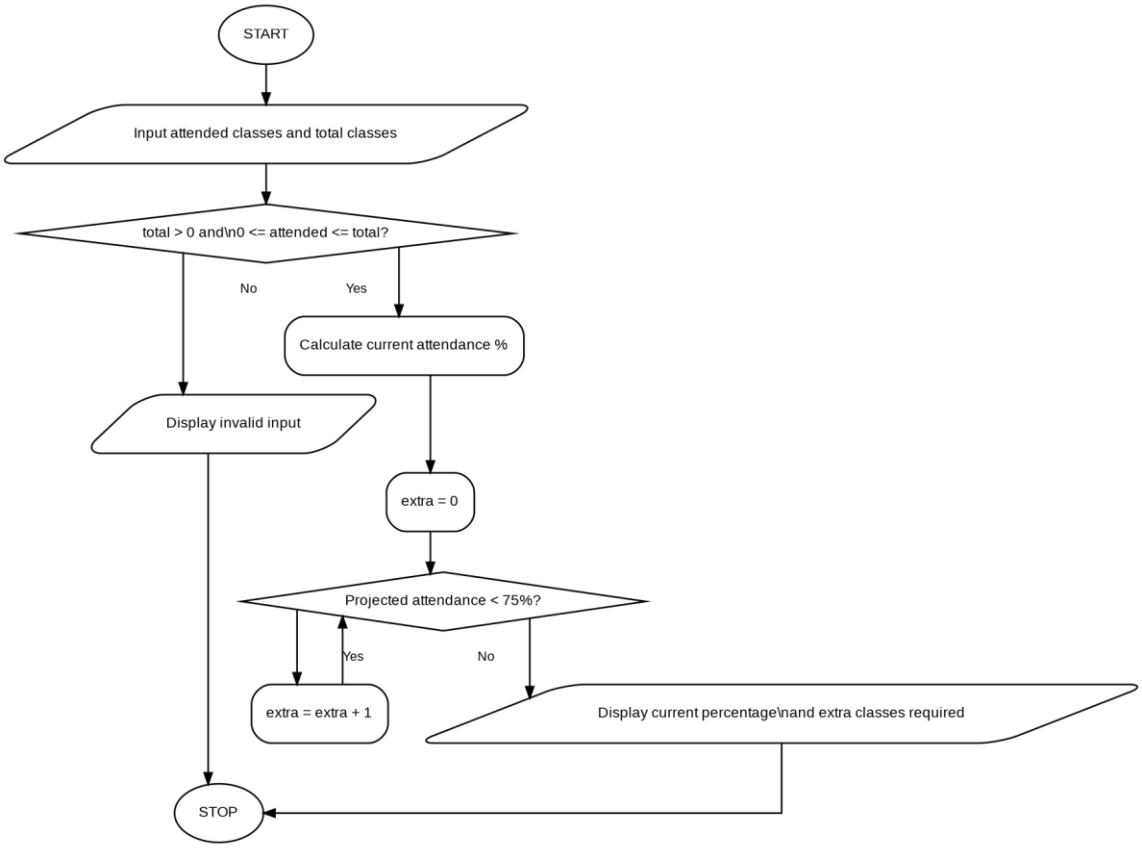
3. Assumptions

- Total classes must be greater than zero.
- Attended classes must be between zero and total classes.
- Every additional class is attended.
- The target attendance is 75%.

4. Algorithm

32. Start.
33. Read attended classes and total classes.
34. Validate the values.
35. Calculate current attendance percentage.
36. Set extra classes to zero.
37. While the projected percentage is below 75%, increase extra by one.
38. Display current percentage and extra classes required.
39. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT attended, total
IF total <= 0 OR attended < 0 OR attended > total THEN DISPLAY "Invalid input"; STOP
percentage ← attended / total × 100
extra ← 0
WHILE (attended + extra) / (total + extra) × 100 < 75
  extra ← extra + 1
DISPLAY percentage, extra
STOP
```

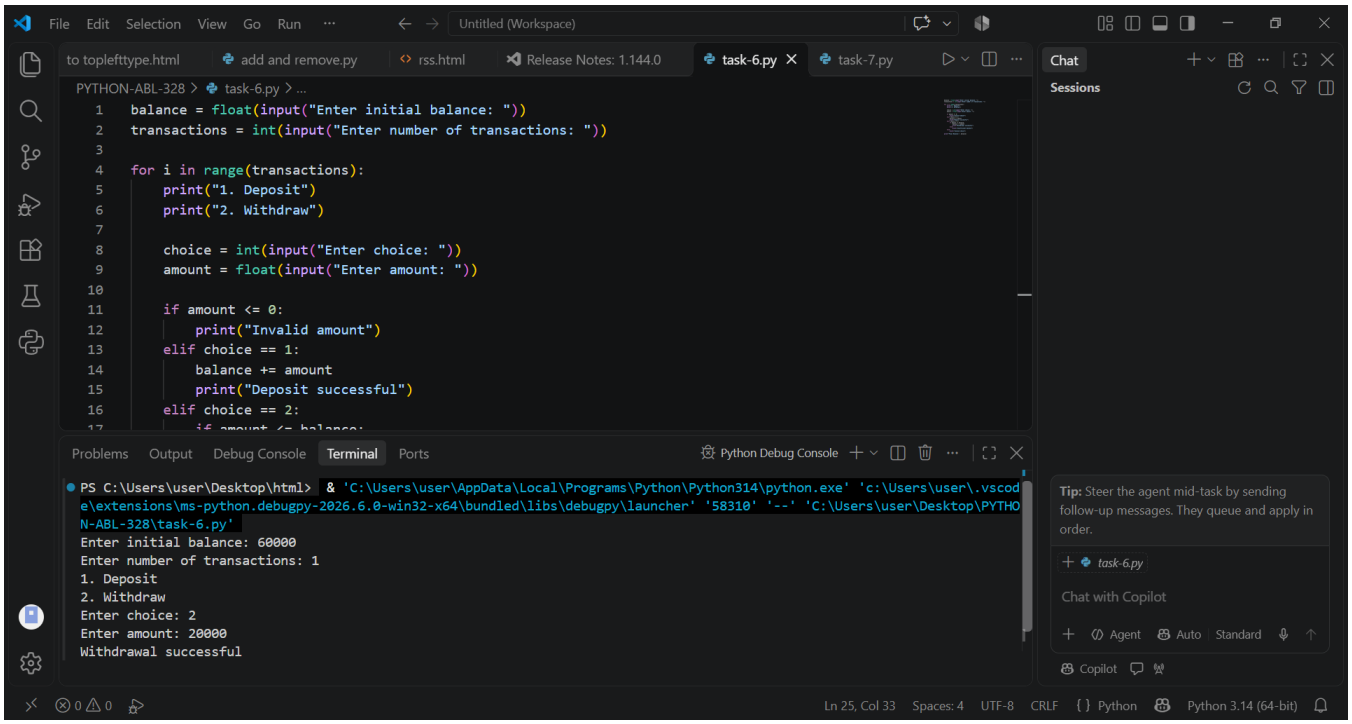
7. Test Cases

Input	Expected result
60 attended, 100 total	Current 60%; 60 additional classes required
75 attended, 100 total	Current 75%; 0 additional classes required
80 attended, 100 total	Current 80%; 0 additional classes required

8. Conclusion

The program calculates attendance and uses repetition to find the minimum number of future attended classes needed to reach the target.

Task-6: ATM Transaction Sequence Simulation



1. Problem Statement

Simulate a sequence of ATM deposits and withdrawals, maintain the account balance, reject invalid amounts and prevent withdrawals greater than the available balance.

2. Objectives

- Read the opening balance and transaction count.
- Process deposits and withdrawals.
- Validate transaction amounts and choices.
- Display the final balance.

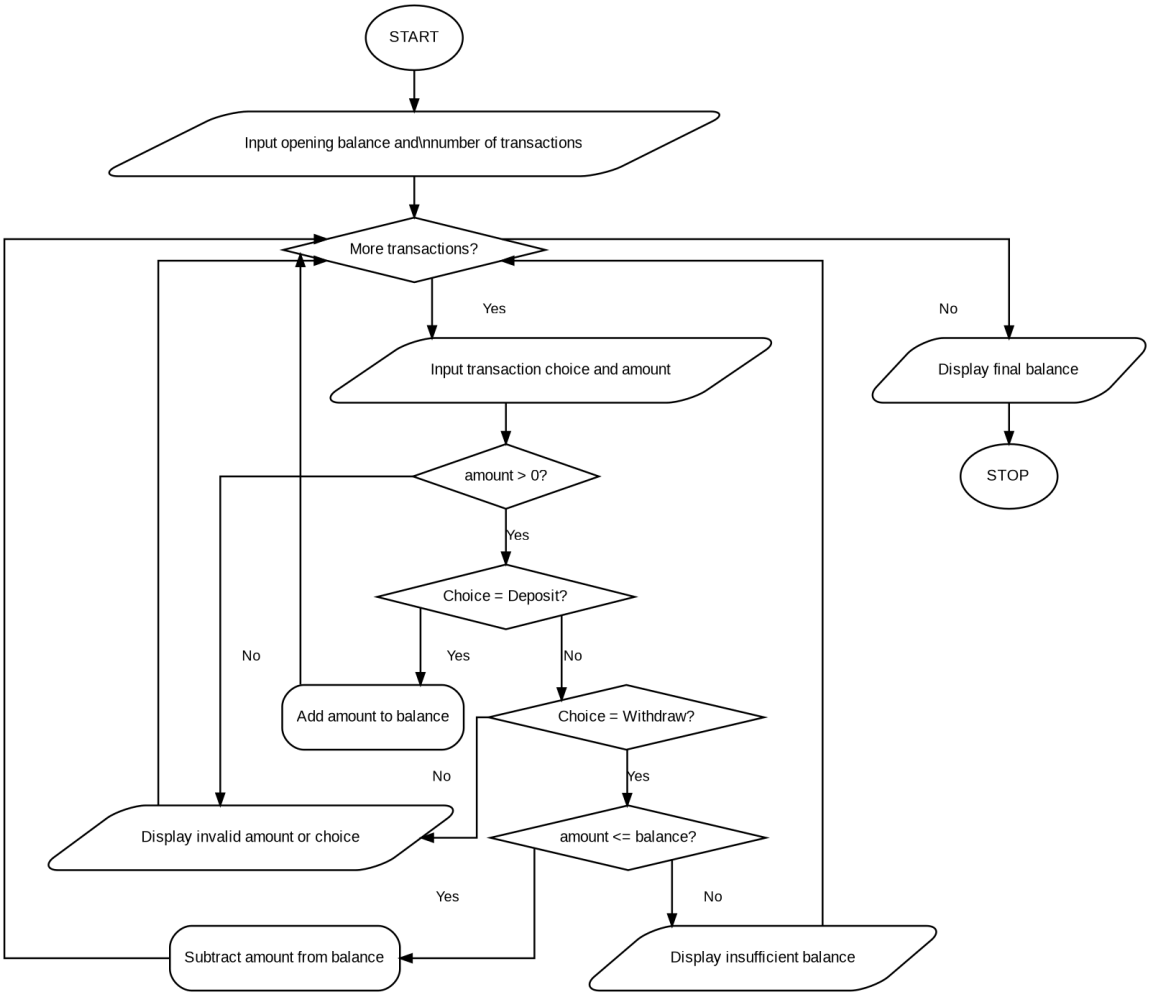
3. Assumptions

- The opening balance is non-negative.
- A transaction amount must be greater than zero.
- Deposit adds to the balance.
- Withdrawal is accepted only when the amount does not exceed the balance.
- Only menu choices 1 and 2 are supported.

4. Algorithm

40. Start.
41. Read initial balance and number of transactions.
42. Repeat for each transaction: display menu; read choice and amount.
43. Reject non-positive amounts.
44. For deposit, add amount to balance.
45. For withdrawal, check balance and subtract if sufficient.
46. Report invalid choices or insufficient balance.
47. Display final balance.
48. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT balance, n
FOR i from 1 to n
  DISPLAY deposit/withdraw menu
  INPUT choice, amount
  IF amount <= 0 DISPLAY "Invalid amount"
  ELSE IF choice = 1 balance ← balance + amount
  ELSE IF choice = 2
    IF amount <= balance balance ← balance – amount
    ELSE DISPLAY "Insufficient balance"
  ELSE DISPLAY "Invalid choice"
END FOR
DISPLAY balance
STOP
```

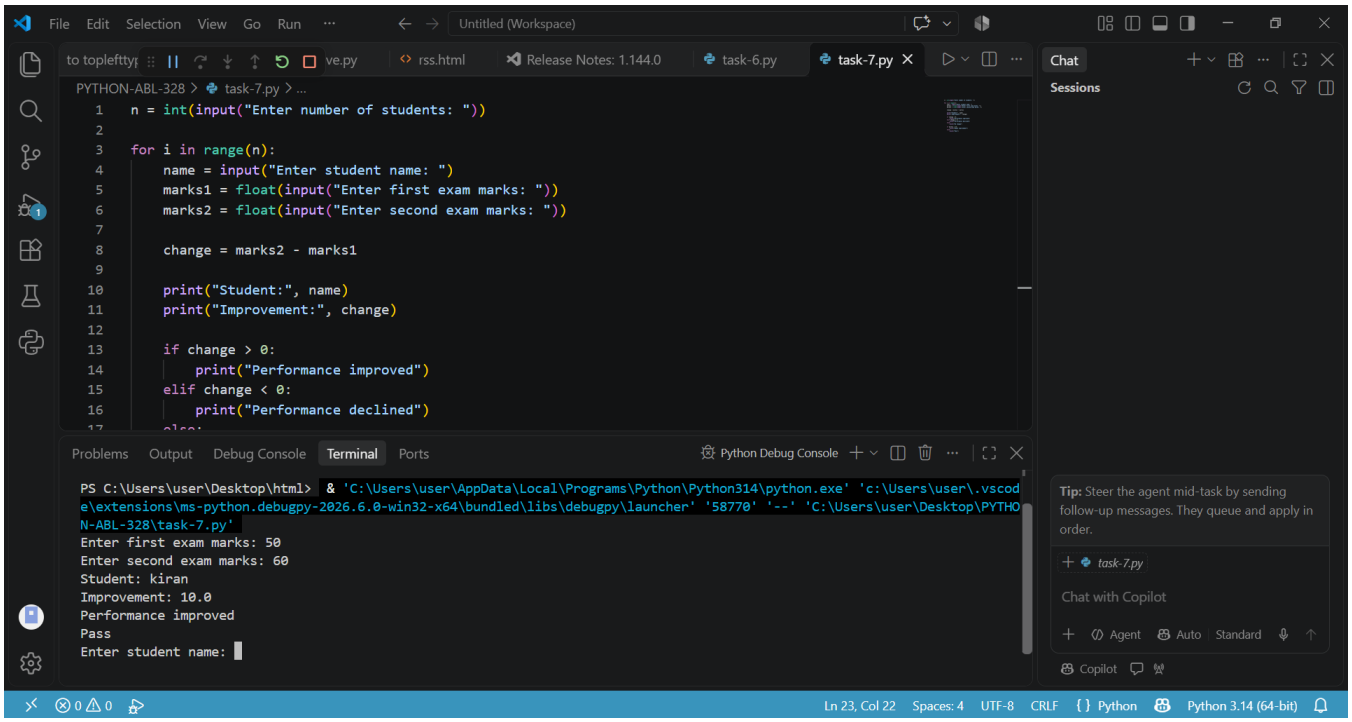
7. Test Cases

Input	Expected result
Opening 1000; deposit 500	Final balance 1500
Opening 1000; withdraw 300	Final balance 700
Opening 1000; withdraw 1200	Withdrawal rejected; final balance 1000
Opening 1000; amount 0	Invalid amount; balance unchanged

8. Conclusion

The program simulates basic ATM transactions using loops and conditional statements while protecting the balance from invalid withdrawals.

Task-7: Student Performance Trend Analysis



1. Problem Statement

Compare each student's marks in two examinations, calculate the change in marks, and report whether performance improved, declined, or remained unchanged.

2. Objectives

- Accept student names and two exam scores.
- Calculate the difference between scores.
- Classify the performance trend.
- Identify whether the second exam score is below the pass mark.

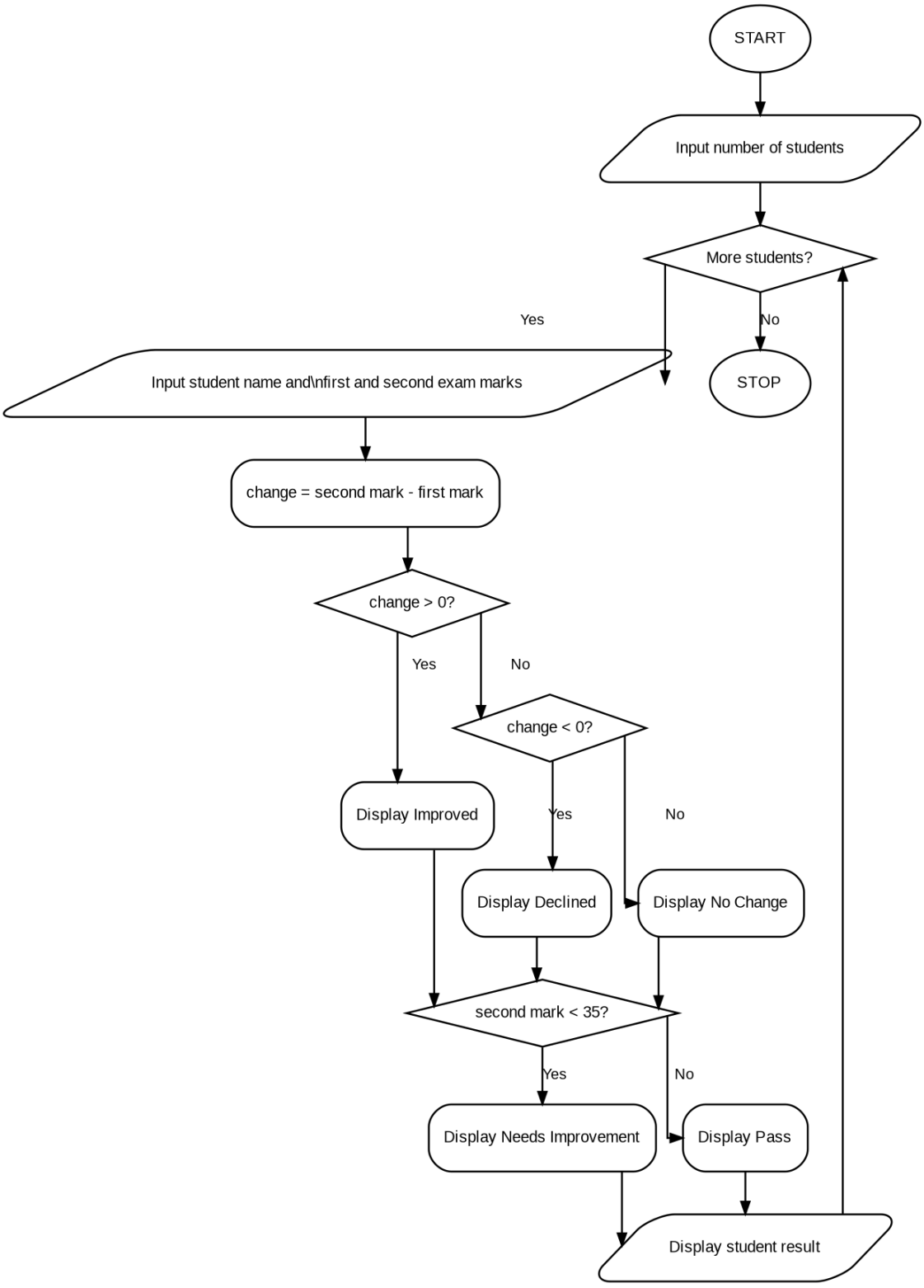
3. Assumptions

- Marks are entered on the same scale, from 0 to 100.
- A positive difference means improvement; a negative difference means decline.
- A second-exam mark below 35 is flagged as needing improvement.

4. Algorithm

49. Start.
50. Read the number of students.
51. For each student, read name and two marks.
52. Calculate change as second mark minus first mark.
53. Classify the trend using the sign of change.
54. Check whether the second mark is below 35.
55. Display the student-wise result.
56. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
FOR each student
  INPUT name, first_mark, second_mark
```

```
    change ← second_mark – first_mark
    IF change > 0 DISPLAY "Improved"
    ELSE IF change < 0 DISPLAY "Declined"
    ELSE DISPLAY "No change"
    IF second_mark < 35 DISPLAY "Needs improvement" ELSE DISPLAY "Pass"
  END FOR
  STOP
```

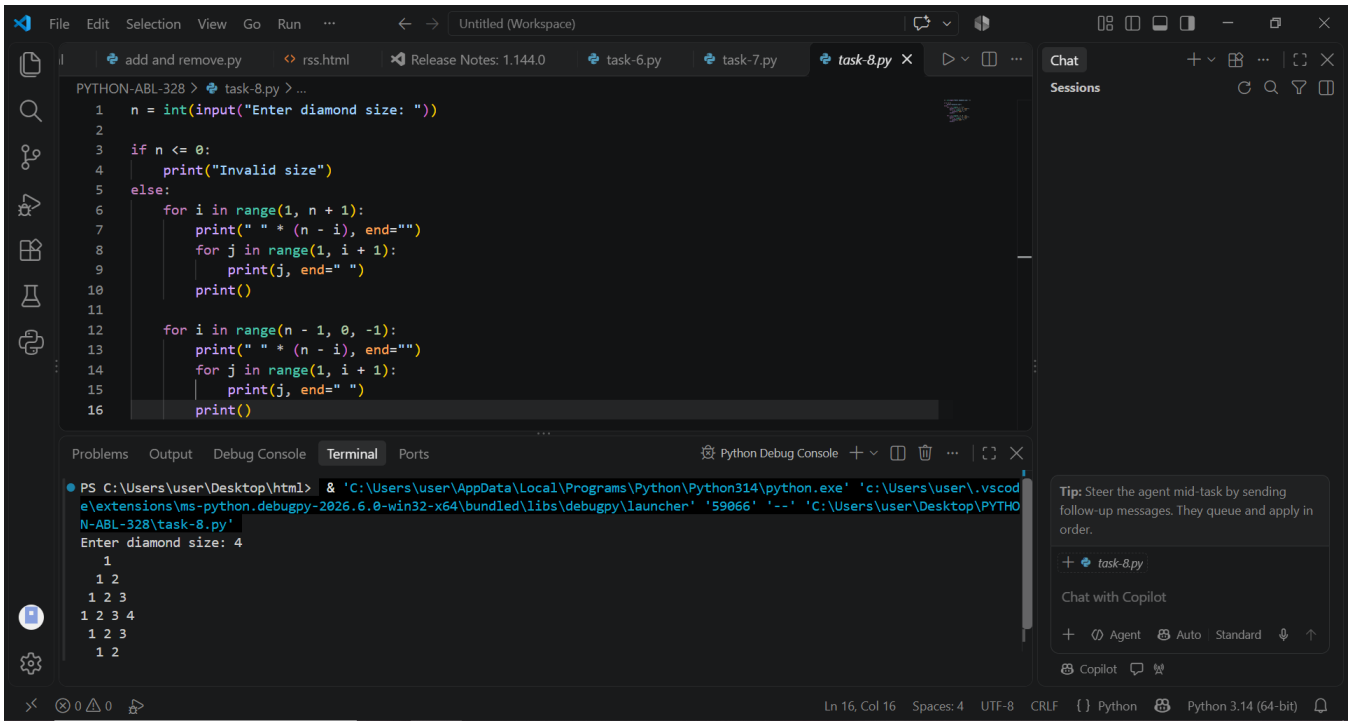
7. Test Cases

Input	Expected result
Asha: 50, 65	Change +15; improved; pass
Bala: 70, 60	Change −10; declined; pass
Charan: 35, 35	Change 0; no change; pass
Diya: 40, 30	Change −10; declined; needs improvement

8. Conclusion

The program compares examination scores and uses conditional statements to describe each student's performance trend.

Task-8: Advanced Numeric Pattern Generator



1. Problem Statement

Generate a symmetric numeric pattern based on a user-provided size, validating that the size is positive.

2. Objectives

- Read the pattern size.
- Validate the input.
- Use nested loops to print the upper and lower halves.
- Produce a symmetric numeric pattern.

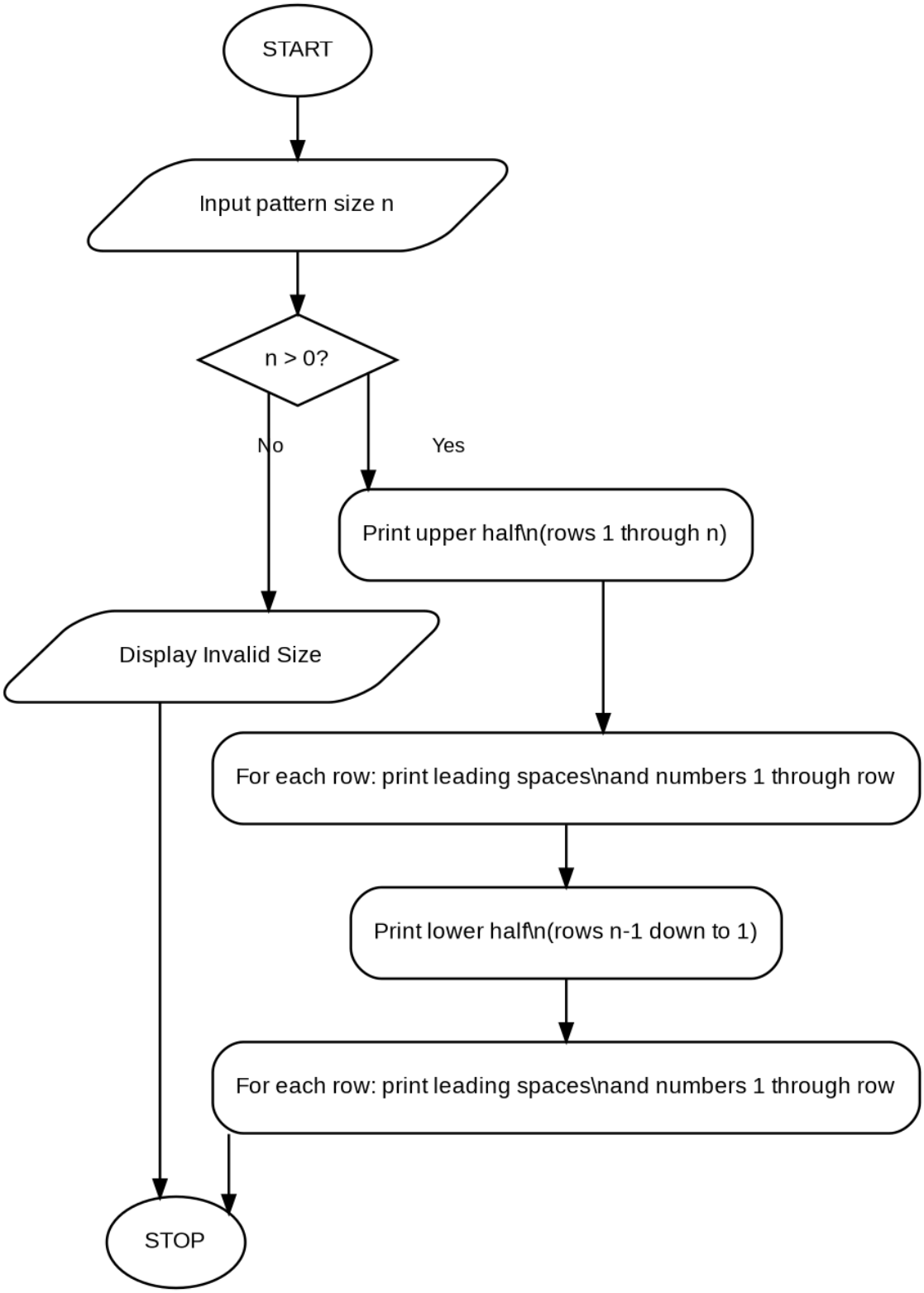
3. Assumptions

- The size is a positive integer.
- The upper half has rows from 1 through n.
- The lower half has rows from n−1 down to 1.
- Each row prints leading spaces followed by ascending numbers separated by spaces.

4. Algorithm

57. Start.
58. Read n.
59. If n is not positive, display an error.
60. For i from 1 to n, print n−i spaces and numbers from 1 to i.
61. For i from n−1 down to 1, print n−i spaces and numbers from 1 to i.
62. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT n
IF n <= 0 THEN DISPLAY "Invalid size"
```

```
ELSE
  FOR i ← 1 TO n
    PRINT (n – i) spaces
    FOR j ← 1 TO i PRINT j
    PRINT newline
  FOR i ← n – 1 DOWNTO 1
    PRINT (n – i) spaces
    FOR j ← 1 TO i PRINT j
    PRINT newline
STOP
```

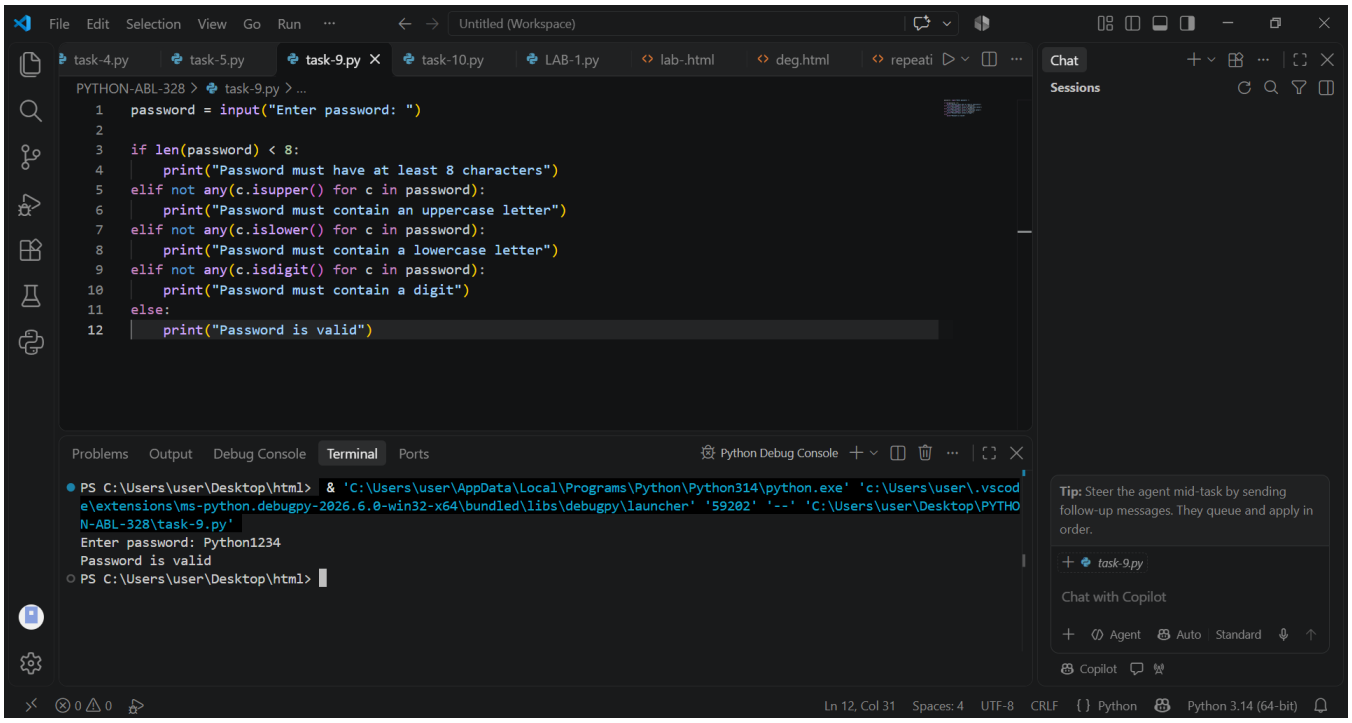
7. Test Cases

Input	Expected result
n = 1	1
n = 3	1 / 1 2 / 1 2 3 / 1 2 / 1
n = 0	Invalid size

8. Conclusion

The program uses nested loops and spacing to generate a symmetric number pattern and rejects invalid dimensions.

Task-9: Password Policy Validation Engine



1. Problem Statement

Validate a sample password string against basic rules for minimum length and the presence of uppercase letters, lowercase letters, and digits.

2. Objectives

- Read a password string.
- Check its length.
- Check for uppercase, lowercase, and numeric characters.
- Display the first failed condition or a valid message.

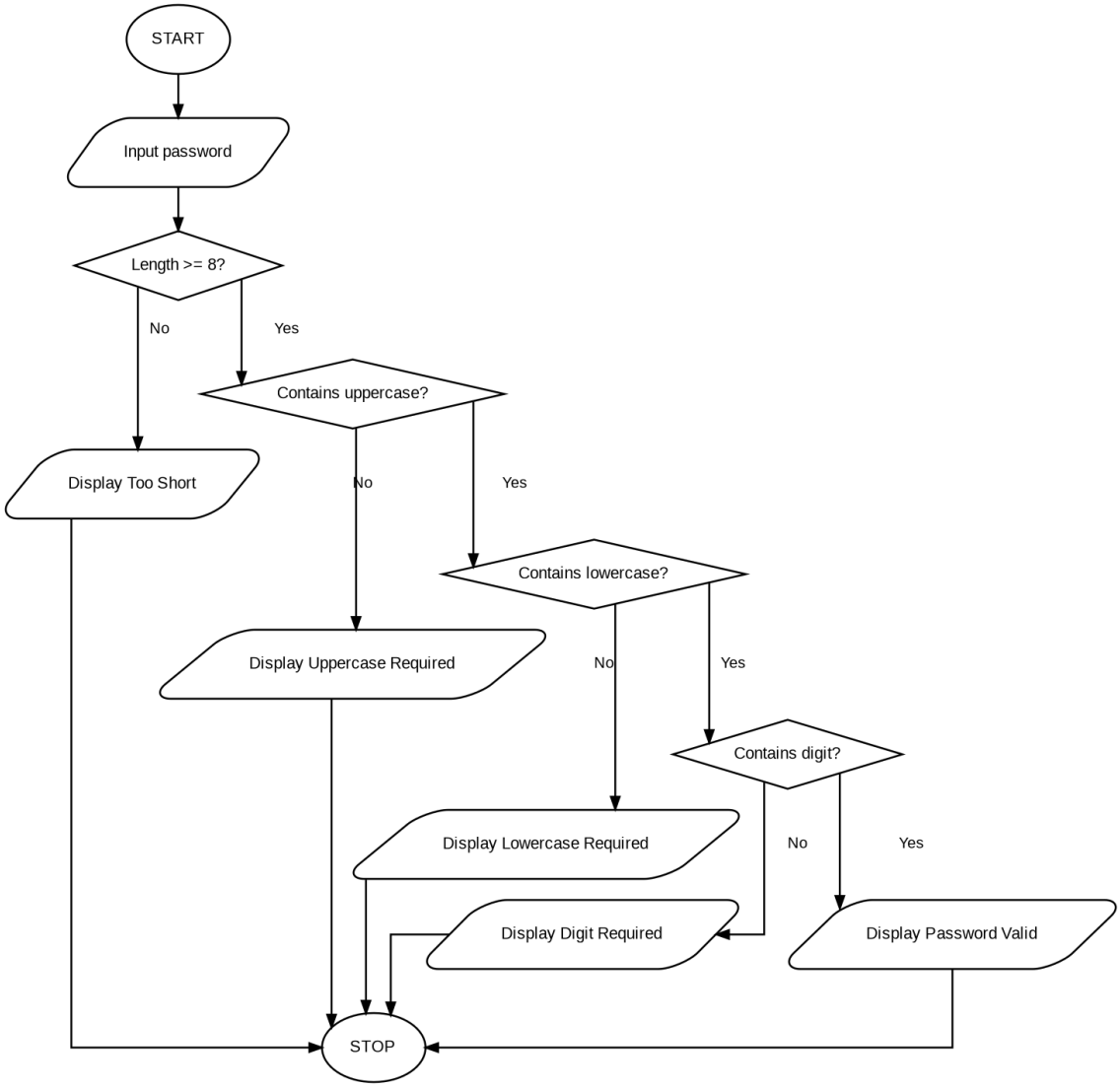
3. Assumptions

- The minimum password length is eight characters.
- At least one uppercase letter, one lowercase letter, and one digit are required.
- The program reports the first failed rule because it uses an if-elif chain.
- This is an educational validation exercise, not a complete security assessment.

4. Algorithm

63. Start.
64. Read the password.
65. Check whether its length is at least eight.
66. If not, display the length error.
67. Otherwise check for uppercase, lowercase, and digit in order.
68. Display the first failed rule or that the password is valid.
69. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT password
IF length(password) < 8 DISPLAY "Too short"
ELSE IF no uppercase character DISPLAY "Uppercase required"
ELSE IF no lowercase character DISPLAY "Lowercase required"
ELSE IF no digit DISPLAY "Digit required"
ELSE DISPLAY "Password is valid"
STOP
```

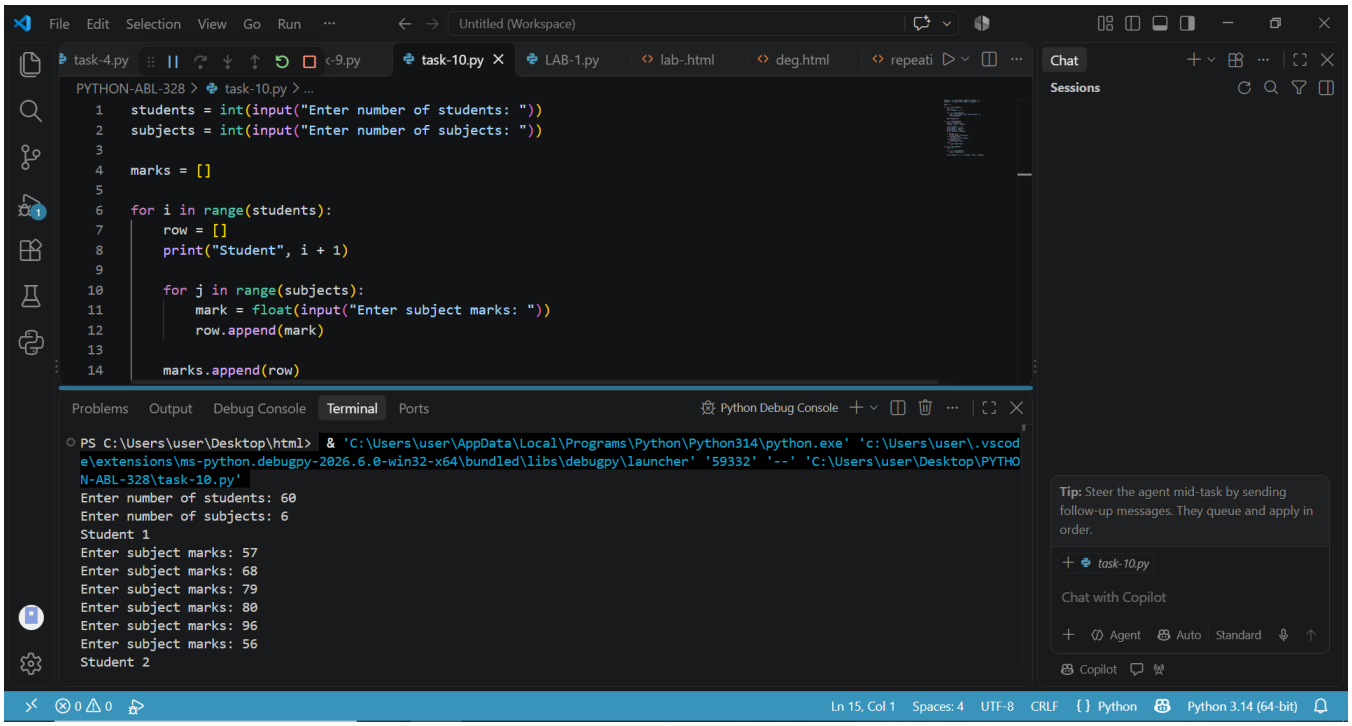
7. Test Cases

Input	Expected result
abc12	Too short
abcdefgh1	Uppercase required
ABCDEFGH1	Lowercase required
Abcdefgh	Digit required
Abcd1234	Password is valid

8. Conclusion

The program applies string checks to validate a password against the stated classroom rules. It checks only the listed conditions.

Task-10: Student Marks Matrix Analysis



1. Problem Statement

Store marks for multiple students and subjects in a nested list, calculate each student's total and average, assign a grade, and calculate each subject's average.

2. Objectives

- Read the number of students and subjects.
- Store marks in a two-dimensional list.
- Calculate row-wise totals and averages.
- Calculate column-wise subject averages.
- Classify students using average marks.

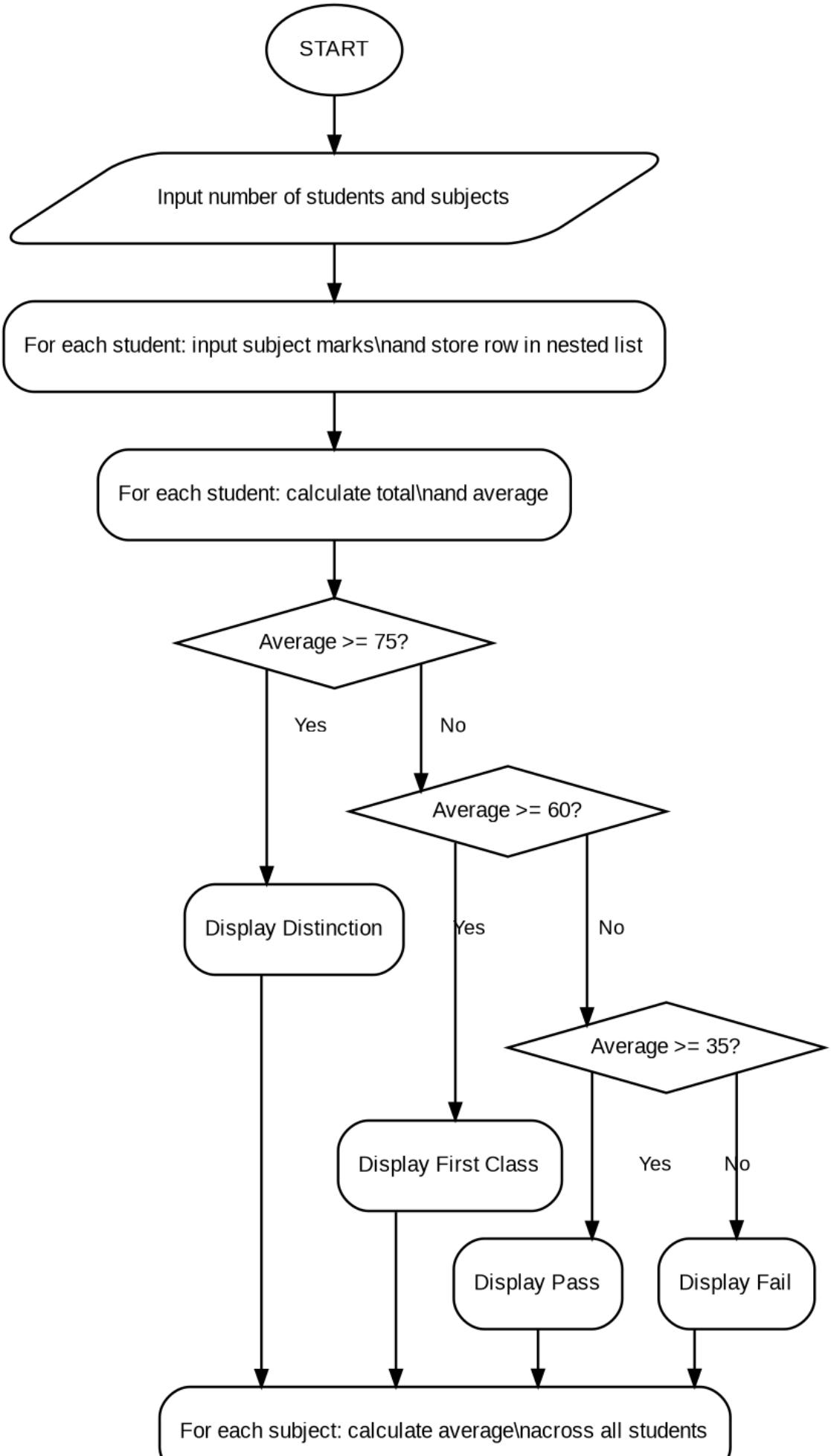
3. Assumptions

- The number of students and subjects must be positive.
- Each mark is between 0 and 100.
- Grade thresholds are: distinction at 75 or above, first class at 60 or above, pass at 35 or above, otherwise fail.
- The simplified program does not apply subject-wise pass rules.

4. Algorithm

70. Start.
71. Read student count and subject count.
72. Create an empty marks matrix.
73. For each student, read one mark per subject and store the row.
74. For each student, calculate total and average and display a grade.
75. For each subject, add marks across all students and calculate the average.
76. Display subject averages.
77. Stop.

5. Flowchart


START
Input number of students and subjects
For each student: input subject marks\nand store row in nested list
For each student: calculate total\nand average
Average >= 75?
Yes
No
Average >= 60?
Display Distinction
Yes
No
Average >= 35?
Display First Class
Yes
No
Display Pass
Display Fail
For each subject: calculate average\nacross all students

6. Pseudocode

```
START
INPUT students, subjects
marks ← empty nested list
FOR each student
  row ← empty list
  REPEAT subjects times: INPUT mark; append mark to row
  append row to marks
FOR each student row
  total ← SUM(row); average ← total / subjects
  DISPLAY total, average
  IF average >= 75 DISPLAY "Distinction"
  ELSE IF average >= 60 DISPLAY "First Class"
  ELSE IF average >= 35 DISPLAY "Pass" ELSE DISPLAY "Fail"
FOR each subject column
  calculate sum of that column / students
  DISPLAY subject average
STOP
```

7. Test Cases

Input	Expected result
Marks matrix [[80,70],[60,50]]	Student averages 75 and 55; subject averages 70 and 60
Marks matrix [[30,40],[50,60]]	Student averages 35 and 55; subject averages 40 and 50
One student, one subject: [[90]]	Student average 90; subject average 90; Distinction

8. Conclusion

The program demonstrates nested lists and nested loops to calculate student-wise and subject-wise statistics and assign a simple grade.

Submission Checklist

- Attach the Python source code for each selected question.
- Run each program and capture its actual output screenshot.
- Check that your screenshots match the test inputs and outputs.
- Replace illustrative assumptions with faculty-prescribed rules, if any.
- Upload the completed work to the GitHub repository specified in the assignment.